Persistent City Directory

Algoritmos e Estruturas de Dados
Problem statement for the first practical assignment, version 1 – 2026-09-18

1 Introduction

This document describes the 1st practical assignment for the AED (Algorithms and Data Structures) course of the 2nd year of the LEI.

The assignment must be completed in **groups of 2 students**. Students can and should clarify all questions with the course instructors, as well as discuss solutions with their classmates, always respecting the principles of the Code of Ethics available on the course webpage on Moodle.

The objective of this assignment is to program in Java the "Persistent City Directory"described in this specification.

2 Application specification

You are required to develop an application that manages a collection of cities. Each city is characterized by its name, the country to which it belongs, and its number of inhabitants (population). A city is uniquely identified by the combination of its name and country.

The number of cities stored in the system may become very large. Therefore, the application must use appropriate data structures to ensure efficient storage and retrieval of information.

The collection of cities must be maintained between executions. When the application starts, it must attempt to restore the city directory from the application's serialization file. If the serialization file exists, the application must deserialize its contents before performing any operation, and writes the message "City directory restored successfully."on console. Otherwise, the application must create an empty directory, and writes messages "City directory not found. Creating an empty city directory."on console.

Before the application terminates, the current state of the city directory must be serialized and stored so that it can be restored during the next execution.

When the application starts, it attempts to load the city directory. If the directory is successfully loaded, the application must display the following message:

```
City directory restored successfully.↵
```

If the directory does not exist, a new empty directory is created and the application must display the following message:

```
City directory not found.↵
Creating an empty city directory.↵
```

After completing this initialization, the application must repeatedly prompt the user for one of the commands described in the following sections

Unless otherwise specified, the application is case-insensitive. You may assume that all input provided by the user is valid, with the correct data types and the required number of input values. Cities names are case-insensitive, meaning that uppercase and lowercase letters are considered equivalent (e.g., Lisbon, LISBON, and lisbon refer to the same city). Although cities names are case-insensitive when entered by the user, the application must always display them using the original capitalization. Country names, however, will always be entered following normal capitalization, meaning the first letter of each word is uppercase and the remaining letters are lowercase (e.g., Canada, United States).

If multiple errors (unsuccessful command responses) are detected while processing an operation, only the first error, according to the order in which the errors are specified in this document, must be reported. No additional error messages should be displayed for that operation.

2.1 Command `help`

Displays the list of available commands together with a brief description of each one.

Input

```
help↵
```

Output

```
addCity:  Adds a new city to the directory.↵
removeCity:  Removes an existing city from the directory.↵
population:  Returns the number of inhabitants of a city.↵
listCities:  Lists all cities currently stored in the directory.↵
help:  Displays the list of available commands.↵
quit:  Terminates the application.↵
```

2.2 Command `addCity`

Adds a new city to the directory.

Input

```
addCity↵
<city_name>↵
<country_name>↵
<population>↵
```

Output

```
City successfully added.↵
```

Unsuccessful response

1. If a city with the same name in the same country already exists:
   `City already exists.↵`

2.3 Command `removeCity`

Removes an existing city from the directory.

Input

```
removeCity↵
<city_name>↵
<country_name>↵
```

Output

```
City successfully removed.↵
```

Unsuccessful response

1. If the city does not exist:
   `City not found.↵`

2.4 Command `population`

Returns the number of inhabitants of a city.

Input

```
population↵
<city_name>↵
<country_name>↵
```

Output

```
Population:  <number_of_inhabitants>↵
```

Unsuccessful response

1. If the city does not exist:
   `City not found.↵`

2.5 Command `listCities`

Lists all cities currently stored in the directory. The cities must be displayed sorted by:

1. Country (ascending alphabetical order);
2. Number of inhabitants (descending order);
3. City name (ascending alphabetical order).

Input

```
listCities↵
```

Output

One city per line in the following format:
`<country> - <population> - <city>↵`

If the directory is empty:
`No cities available.↵`

2.6 Command `quit`

Terminates the application. Before exiting, the application must serialize the current city directory so that it can be restored during the next execution.

Input

```
quit↵
```

Output

```
City directory saved.↵
Goodbye.↵
```

3 Example Program Execution

In the examples below, blue text represents the input entered by the user in the console, while **black text** represents the output produced by the application.

The following example illustrates a complete first execution of the application.

```
City directory not found.↵
Creating an empty city directory.↵
addCity↵
Lisbon↵
Portugal↵
545923↵
City successfully added.↵
addCity↵
Porto↵
Portugal↵
231962↵
City successfully added.↵
addCity↵
Madrid↵
Spain↵
3280782↵
City successfully added.↵
addCity↵
New York↵
United States↵
3480782↵
City successfully added.↵
addCity↵
Barcelona↵
Spain↵
1664182↵
City successfully added.↵
addCity↵
Paris↵
France↵
2102650↵
City successfully added.↵
```

addCity↵
BErlin↵
Germany↵
3677472↵
City successfully added.↵
addCity↵
Munich↵
Germany↵
1488202↵
City successfully added.↵
addCity↵
Lisbon↵
Portugal↵
54923↵
City already exists.↵
population↵
Madrid↵
SpAin↵
Population: 3280782↵
population↵
RoME↵
Italy↵
City not found.↵
listCities↵
France - 2102650 - Paris↵
Germany - 3677472 - BErlin↵
Germany - 1488202 - Munich↵
Portugal - 545923 - Lisbon↵
Portugal - 231962 - Porto↵
United States - 3480782 - New York↵
Spain - 3280782 - Madrid↵
Spain - 1664182 - Barcelona↵
removeCity↵
Munich↵
Germany↵
City successfully removed.↵
removeCity↵
New YORK↵
United States↵
City successfully removed.↵
removeCity↵
Rome↵
Italy↵
City not found.↵
listCities↵
France - 2102650 - Paris↵
Germany - 3677472 - BErlin↵
Portugal - 545923 - Lisbon↵
Portugal - 231962 - Porto↵
Spain - 3280782 - Madrid↵
Spain - 1664182 - Barcelona↵
quit↵
City directory saved.↵
Goodbye.↵

Assume that the application is executed again.

```
City directory restored successfully.↵
population↵
Berlin↵
Germany↵
Population: 3677472↵
listCities↵
France - 2102650 - Paris↵
Germany - 3677472 - BErlin↵
Portugal - 545923 - Lisbon↵
Portugal - 231962 - Porto↵
Spain - 3280782 - Madrid↵
Spain - 1664182 - Barcelona↵
quit↵
City directory saved.↵
Goodbye.↵
```

4 Assessment

This is the first programming assignment of the course and contributes 5% to the final grade.

The assignment must be completed in **groups of 2 students**, and must be submitted via `Mooshak` (contest "AED_26_TP1"). The **Mooshak identifier** must be the concatenation of the student numbers, separated by an underscore (_), with the smallest number coming first.

The program must be submitted no later than **[date]** at **[time]**. Submissions received after this deadline will not be evaluated.

Only assignments with at least one Mooshak submission that achieves a score greater than or equal to 90 points ($\geqslant 90$) will be considered for evaluation.

The implementation must **not use the "java.util"** package. Students must use only the classes and interfaces available in the supplied "**dataStructures**"package. When using the "dataStructures"package, students must first remove all classes and interfaces that are not required for their implementation. The remaining interfaces and classes provided in the package **must not be modified**. Students may only implement the methods marked as exercises and add new private methods whenever necessary. **Only interfaces and classes effectively used by the application should remain in the package.**

In the documentation of each class, you must specify its **space complexity**. In the documentation of each class method, you must specify its **time complexity** (best and worst case, whenever applicable).